UNIT III

Trees and Graphs

Notes written to answer the exam questions

Question	Marks	Where to read
Q1. Binary search tree: insertion and deletion with diagrams	10	Section 3
Q2. Preorder, inorder and postorder traversals with an example	10	Section 2
Q3. Heap trees and the heap sort method	5	Section 4
Q4. Basic binary-tree terminology with a diagram	5	Section 1
Q5. Graphs, types, BFS and DFS with algorithms and an example	10	Sections 5 and 6
Q6. Graph representation: adjacency matrix and adjacency list	5	Section 7
Q7. Height and degree of a tree node	3	Section 1
Q8. Define a root node	2	Section 1
Q9. Construct a BST from 50, 30, 70, 20, 40, 60, 80 and write the traversals	10	Section 3.6

PART A: TREES

1. Basic Tree Terminology

Answers Q4 (5 marks), Q7 (3 marks) and Q8 (2 marks)

Definition: A **tree** is a non-linear, hierarchical data structure made of **nodes** connected by **edges**. It has one special node called the **root**, and the remaining nodes are divided into sub-trees. There is exactly one path between any two nodes, and there is no cycle.

Definition: A **binary tree** is a tree in which every node has **at most two children**, called the **left child** and the **right child**.

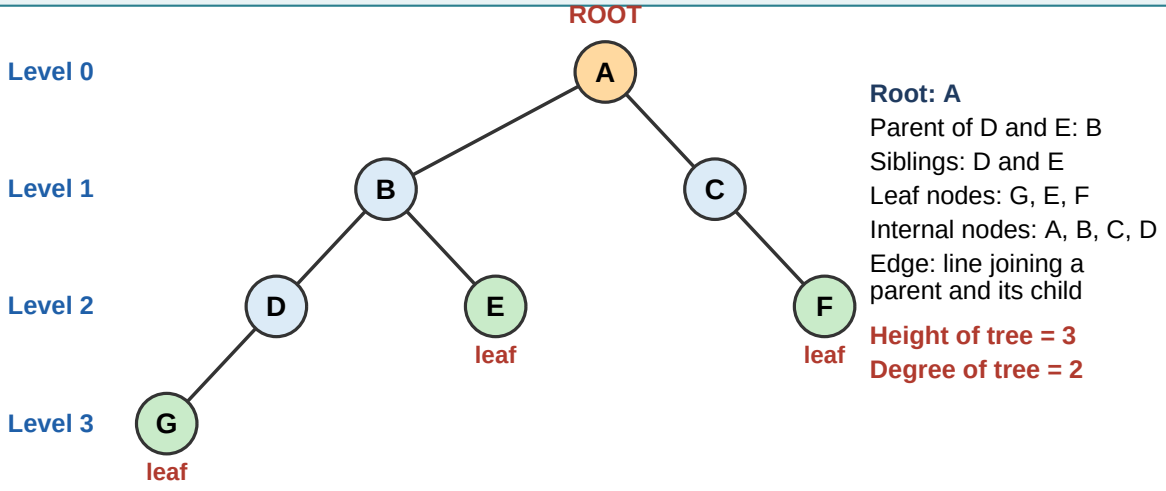


Figure 1: A binary tree with its terminology

Terms

Term	Meaning
Node	An element of the tree; it stores data and links to its children.
Edge	The link (line) joining a parent node to its child.
Root	The topmost node of the tree; it has no parent.
Parent / Child	A node that has a node directly below it is its **parent**; the node directly below is its **child**.
Siblings	Nodes that have the same parent.
Leaf (terminal node)	A node with no children (degree 0).
Internal node (non-terminal)	A node that has at least one child.
Ancestor / Descendant	All nodes on the path from the root to a node are its ancestors; all nodes below a node are its descendants.
Path	A sequence of nodes joined by edges. Path length = number of edges.
Degree of a node	The number of children (sub-trees) of the node.
Degree of a tree	The maximum degree among all nodes. For a binary tree it is at most 2.
Level (depth) of a node	Number of edges from the root to the node. The root is at level 0.
Height of a node	Number of edges on the longest path from the node down to a leaf. A leaf has height 0.
Height of a tree	The height of the root = number of levels minus 1.
Sub-tree	A node together with all its descendants.

Note: some textbooks count levels and height from 1 instead of 0. Follow your textbook and state the convention you use.

Values for the tree in Figure 1

Node	Parent	Children	Degree	Level	Height
A	-	B, C	2	0	3
B	A	D, E	2	1	2
C	A	F	1	1	1
D	B	G	1	2	1
E	B	-	0	2	0
F	C	-	0	2	0
G	D	-	0	3	0

Q8: Root node (2 marks): The **root node** is the **topmost node** of a tree. It has **no parent**, a tree has only **one** root, and every other node can be reached from it by following edges downward. It is the starting point of every traversal. In Figure 1, A is the root.

Q7: Height and degree of a node (3 marks): Degree of a node is the **number of children** (sub-trees) that the node has. **Height of a node** is the **number of edges on the longest path from that node down to a leaf**; the height of a leaf is 0. In Figure 1: degree(B) = 2, degree(C) =

1, degree(E) = 0; height(B) = 2 (path B-D-G), height(C) = 1, height(G) = 0.

2. Binary Tree Traversals

Answers Q2 (10 marks)

Definition: Tree traversal is the process of **visiting every node of a tree exactly once** in a systematic order. For a binary tree there are three depth-first traversals: **preorder**, **inorder** and **postorder**. Let N = root (node), L = left sub-tree, R = right sub-tree.

Traversal	Order	Rule to remember
Preorder	N, L, R	Root is visited **first**
Inorder	L, N, R	Root is visited in the **middle**
Postorder	L, R, N	Root is visited **last**

Algorithm: Preorder (root, left, right)
1. If the tree is empty, stop.
2. Visit the root node.
3. Traverse the left sub-tree in preorder.
4. Traverse the right sub-tree in preorder.

Algorithm: Inorder (left, root, right)
1. If the tree is empty, stop.
2. Traverse the left sub-tree in inorder.
3. Visit the root node.
4. Traverse the right sub-tree in inorder.

Algorithm: Postorder (left, right, root)
1. If the tree is empty, stop.
2. Traverse the left sub-tree in postorder.
3. Traverse the right sub-tree in postorder.
4. Visit the root node.

Example

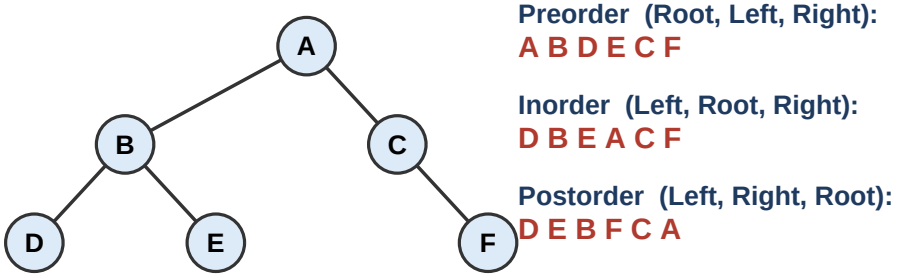


Figure 2: A binary tree and its three traversals

Preorder: visit A; go left to B and visit B; go left to D and visit D; D has no children, so back to B and go right: visit E; back to A and go right: visit C; C has no left child; go right: visit F. Result: **A**

B D E C F.

Inorder: go to the leftmost node D and visit it; then its parent B; then B's right child E; then the root A; then go right: C has no left child, visit C; then F. Result: **D B E A C F**.

Postorder: visit the left sub-tree of B (D, then E), then B itself; then the right sub-tree of A (F, then C); and the root A last. Result: **D E B F C A**.

Traversal	Common use
Preorder	Making a copy of a tree; getting the prefix form of an expression
Inorder	In a binary search tree it gives the keys in **sorted (ascending) order**
Postorder	Deleting a tree (children before parent); getting the postfix form of an expression

Time complexity: each traversal visits every node once, so it takes **O(n)** time.

3. Binary Search Tree (BST)

Answers Q1 (10 marks) and Q9 (10 marks)

3.1 Definition and properties

Definition: A **binary search tree** is a binary tree in which, for every node, all the keys in its **left sub-tree are smaller** than the node's key and all the keys in its **right sub-tree are larger**. (Duplicate keys are normally not allowed.)

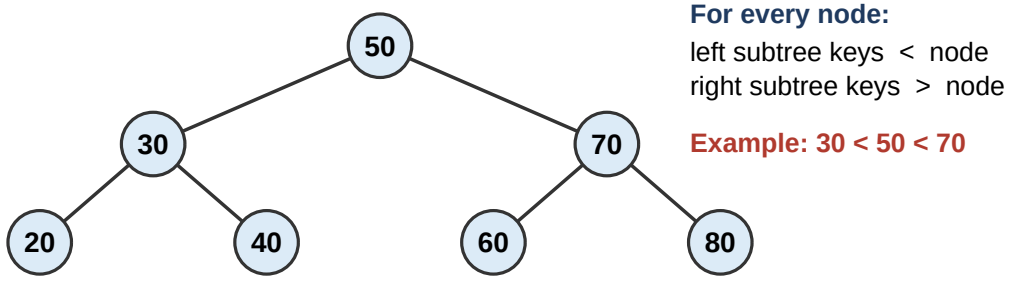


Figure 3: A binary search tree

- The rule 'left < node < right' holds at **every** node, not only at the root.
- The **inorder traversal** of a BST gives the keys in ascending order (here: 20, 30, 40, 50, 60, 70, 80).
- The smallest key is the leftmost node; the largest key is the rightmost node.
- Search, insertion and deletion follow a single path from the root downward.

3.2 Searching

Algorithm: Search in a BST
1. Start at the root.
2. If the current node is empty, the key is not present; stop.
3. If the key equals the node's key, the search is successful; stop.

4. If the key is smaller than the node's key, move to the left child; otherwise move to the right child.
5. Repeat from step 2.

3.3 Insertion

Definition: Insertion places a new key in the tree so that the BST property is kept. A new key is always inserted as a **leaf**.

Algorithm: Insert a key in a BST
1. If the tree is empty, make the new node the root; stop.
2. Start at the root and compare the new key with the current node.
3. If the new key is smaller, move to the left child; if it is larger, move to the right child. (If equal, reject the duplicate.)
4. Repeat step 3 until an empty position (null link) is reached.
5. Attach the new node at that empty position as a leaf.

Example: insert 45 into the tree of Figure 3. Compare with 50: 45 < 50, go left. Compare with 30: 45 > 30, go right. Compare with 40: 45 > 40, go right. The right child of 40 is empty, so 45 becomes the **right child of 40**.

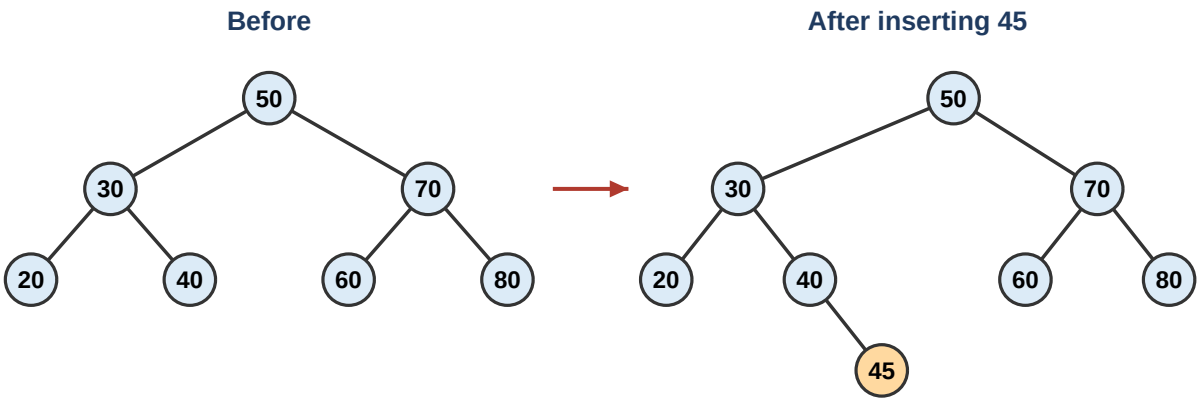


Figure 4: Insertion of 45 in a BST

3.4 Deletion

Definition: Deletion removes a key from the tree so that the BST property is kept. There are **three cases**, depending on the number of children of the node to be deleted.

Algorithm: Delete a key from a BST
1. Search for the node that contains the key. If it is not found, stop.
2. **Case 1: the node is a leaf.** Remove the node and set its parent's link to null.
3. **Case 2: the node has one child.** Remove the node and connect its parent directly to its child.
4. **Case 3: the node has two children.** Find its **inorder successor** (the smallest node in its right sub-tree; the inorder predecessor, i.e. the largest node of the left sub-tree, can also be used). Copy the successor's key into the node, then delete the successor node (it has at most one child) by case 1 or case 2.

Examples below use the tree of Figure 4 (after inserting 45).

Case 1: delete 20 (a leaf). The node is simply removed; the left link of 30 becomes null.

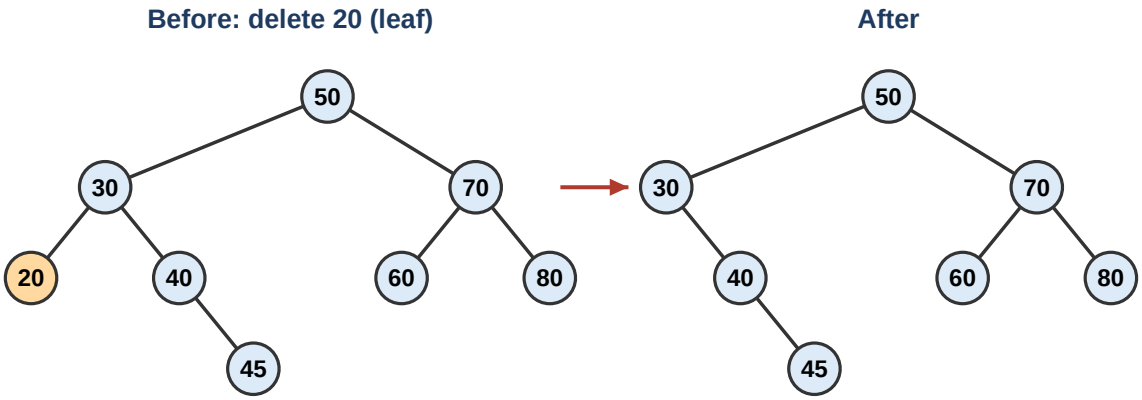


Figure 5: Deleting a leaf node (20)

Case 2: delete 40 (it has one child, 45). Node 40 is removed and its child 45 is attached to 40's parent, 30.

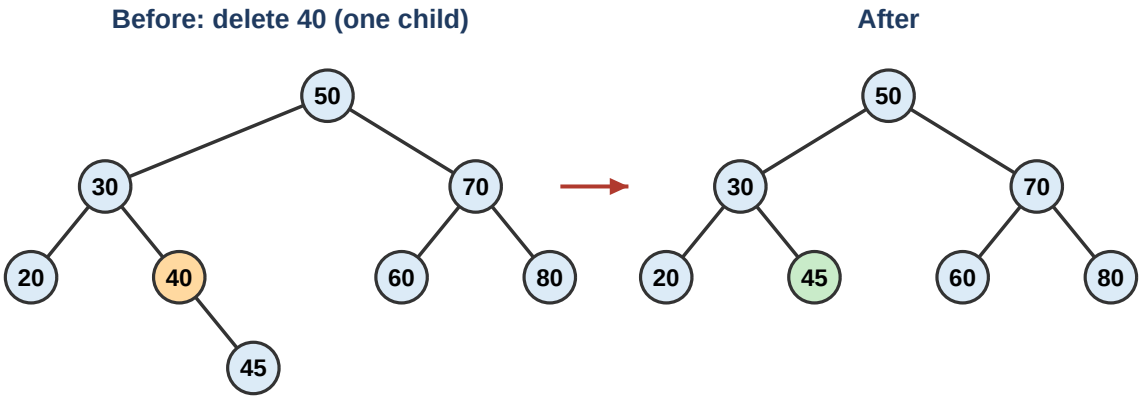


Figure 6: Deleting a node with one child (40)

Case 3: delete 50, the root (it has two children). The inorder successor is the smallest key in the right sub-tree: from 70 go left to 60, which has no left child, so the successor is 60. Copy 60 into the root position and delete the old node 60 (a leaf).

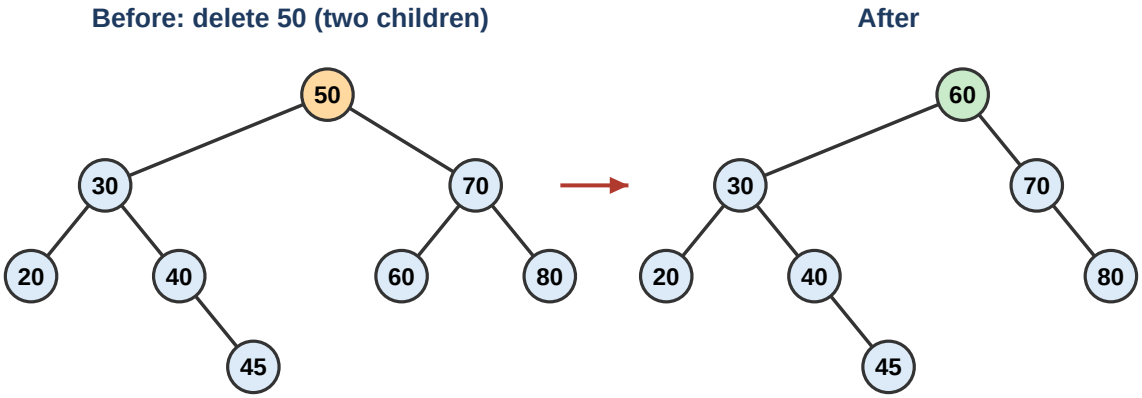


Figure 7: Deleting a node with two children (50)

The inorder traversal after each deletion is still sorted, which confirms that the BST property is kept: 20 30 40 45 60 70 80 (after deleting 50).

3.5 Complexity, advantages and disadvantages

Operation	Average case (balanced tree)	Worst case (skewed tree)
Search	O(log n)	O(n)
Insertion	O(log n)	O(n)
Deletion	O(log n)	O(n)

The worst case occurs when keys are inserted in sorted order (for example 10, 20, 30, 40): the tree becomes a straight line like a linked list. The height of the tree decides the time. Space is O(n).

Advantages	Disadvantages
Search, insertion and deletion are fast (O(log n) on average)	Can become **skewed**, giving O(n) time
Inorder traversal gives sorted data	No automatic balancing
Dynamic size; easier to insert and delete than in a sorted array	Extra memory for the two links in each node
Simple to implement	Duplicate keys are not allowed in the basic form

3.6 Solved problem (Q9): construct a BST and write the traversals

Question: Construct a BST using the keys 50, 30, 70, 20, 40, 60, 80 (in this order) and write its preorder, inorder and postorder traversals.

Solution. Insert the keys one by one. The first key becomes the root; every other key is compared from the root and moved left (smaller) or right (larger) until an empty place is found.

Key	Comparisons	Position
50	Tree is empty	Becomes the root
30	30 < 50: go left	Left child of 50
70	70 > 50: go right	Right child of 50
20	20 < 50: left; 20 < 30: left	Left child of 30
40	40 < 50: left; 40 > 30: right	Right child of 30
60	60 > 50: right; 60 < 70: left	Left child of 70
80	80 > 50: right; 80 > 70: right	Right child of 70

The final BST is the tree shown in Figure 3 (root 50; left sub-tree 30 with children 20 and 40; right sub-tree 70 with children 60 and 80). Its height is 2.

Traversal	Result
Preorder (Root, Left, Right)	**50 30 20 40 70 60 80**
Inorder (Left, Root, Right)	**20 30 40 50 60 70 80** (sorted order)
Postorder (Left, Right, Root)	**20 40 30 60 80 70 50**

4. Heap Tree and Heap Sort

Answers Q3 (5 marks)

Definition: A **heap tree** (heap) is a **complete binary tree** that satisfies the **heap property**. A complete binary tree has every level full except possibly the last, and the last level is filled from left to right.

- **Max-heap:** every parent is **greater than or equal to** its children, so the root is the **largest** element.
- **Min-heap:** every parent is **less than or equal to** its children, so the root is the **smallest** element.

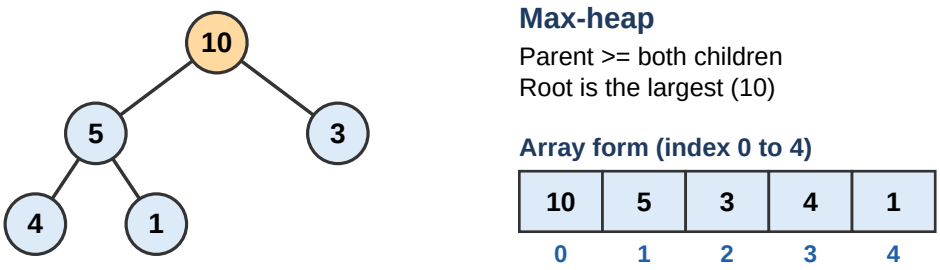


Figure 8: A max-heap and its array form

Array representation (index from 0): for the node at index i, the left child is at 2i + 1, the right child is at 2i + 2, and the parent is at (i - 1) / 2 (integer part). Because the tree is complete, no space is wasted in the array.

Operations on a heap (max-heap)

Algorithm: Insertion in a max-heap
1. Add the new element at the end of the heap (the next free position, so the tree stays complete).
2. Compare it with its parent. If it is greater than the parent, swap them.
3. Repeat step 2 until the parent is greater or the element reaches the root.

Example: insert 60 into the max-heap 50 30 40 10 20. Add at the end: 50 30 40 10 20 60. Its parent is 40, and 60 > 40, so swap: 50 30 60 10 20 40. Now its parent is 50, and 60 > 50, so swap: 60 30 50 10 20 40. The result is the max-heap 60 30 50 10 20 40.

Algorithm: Deletion of the root in a max-heap
1. Remove the root (the largest element).
2. Move the last element of the heap to the root position.
3. Compare it with its children; if it is smaller than the larger child, swap it with that child.
4. Repeat step 3 until the heap property is restored.

Example: delete the root of 60 30 50 10 20 40. Remove 60 and move the last element 40 to the root: 40 30 50 10 20. The larger child of 40 is 50, so swap: 50 30 40 10 20. The heap property is

restored.

Insertion and deletion each take **O(log n)** time (the height of the heap is log n).

Heap sort

Definition: Heap sort is a comparison-based sorting method that uses a **max-heap**. First the array is converted into a max-heap; then the largest element (the root) is repeatedly **moved to the end** of the array and the remaining elements are rebuilt into a heap.

Algorithm: Heap Sort
1. **Build a max-heap** from the array (starting from the last non-leaf node and moving up to the root, push each element down to its correct place).
2. Swap the root (the largest element) with the last element of the heap. The largest element is now in its final position.
3. Reduce the size of the heap by 1 (the last element is now part of the sorted portion).
4. Restore the max-heap by pushing the new root down (heapify).
5. Repeat steps 2 to 4 until only one element is left in the heap. The array is now sorted in ascending order.

Example: sort 4, 10, 3, 5, 1. Step 1 builds the max-heap: **10 5 3 4 1**. The bar | separates the heap part from the sorted part.

Pass	After swapping root with last element	After restoring the heap
1	1 5 3 4 | 10	5 4 3 1 | 10
2	1 4 3 | 5 10	4 1 3 | 5 10
3	3 1 | 4 5 10	3 1 | 4 5 10
4	1 | 3 4 5 10	1 | 3 4 5 10

Final sorted array: **1 3 4 5 10**.

Case	Time	Reason
Best / Average / Worst	O(n log n)	Building the heap takes O(n); each of the n-1 removals takes O(log n)

Space: O(1) (in-place). **Stable:** No.

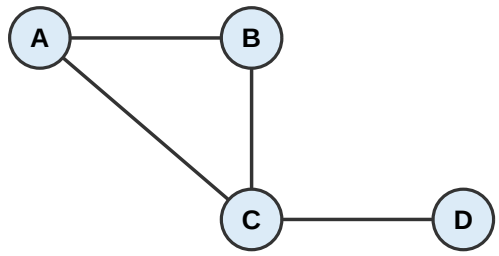
Advantages	Disadvantages
O(n log n) even in the worst case	Not stable (equal elements may change order)
In-place: no extra memory	Usually slower in practice than quick sort and merge sort
The heap is also used as a priority queue	Poor use of the cache; more complex than simple sorts

PART B: GRAPHS

5. Graphs and Types of Graphs

Answers Q5 (10 marks), first part

Definition: A **graph** is a non-linear data structure made of a set of **vertices** (nodes) and a set of **edges** that connect pairs of vertices. It is written as **G = (V, E)**, where V is the set of vertices and E is the set of edges.



G = (V, E)

V = {A, B, C, D}

E = {AB, BC, AC, CD}

Figure 9: A graph with 4 vertices and 4 edges

Term	Meaning
Vertex (node)	A point of the graph.
Edge	A line connecting two vertices.
Neighbour (adjacent vertex)	A vertex joined directly to another vertex by an edge.
Degree	Number of edges connected to a vertex.
In-degree / Out-degree	In a directed graph: edges coming into / going out of a vertex.
Path / Cycle	A sequence of vertices joined by edges / a path that starts and ends at the same vertex.

Types of graphs

1. Based on edges

Definition: Undirected graph: the edges have **no direction**; edge A-B is the same as B-A (two-way). **Directed graph (digraph):** every edge has a **direction** shown by an arrow; A→B allows movement only from A to B (one-way).

2. Based on weight

Definition: Weighted graph: every edge has a number called its **weight** (distance, cost, time). **Unweighted graph:** the edges have **no weights**.

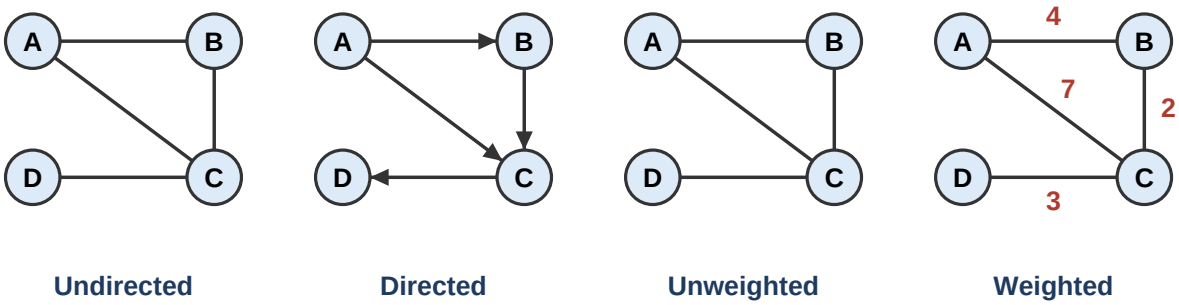


Figure 10: Undirected, directed, unweighted and weighted graphs

3. Cyclic and acyclic. A **cycle** is a path that starts and ends at the same vertex.

Definition: Cyclic graph: contains **at least one cycle**. **Acyclic graph:** contains **no cycle** (a directed acyclic graph is called a DAG; a tree is an acyclic graph).

Figure 11: Cyclic and acyclic graphs

6. Graph Traversals: BFS and DFS

Answers Q5 (10 marks), second part

Definition: Graph traversal is the process of **visiting every vertex** of a graph exactly once, starting from a given vertex. A graph may have cycles, so every visited vertex is **marked** to avoid visiting it again. The two methods are **BFS** and **DFS**.

The same graph is used for both examples (Figure 12). Its adjacency lists are: A: B, C B: A, D C: A, D, E D: B, C, F E: C, F F: D, E. Neighbours are taken in alphabetical order and the start vertex is A.

6.1 Breadth First Search (BFS)

Definition: BFS starts at a given vertex, visits it, then visits **all its neighbours**, then the neighbours of those neighbours, and so on, **level by level**. It uses a **queue** (First In First Out).

Algorithm: BFS

1. Mark the start vertex as visited and insert it into the queue.
2. Repeat steps 3 and 4 while the queue is not empty.
3. Remove the vertex at the front of the queue and visit it.
4. Insert all its **unvisited** neighbours into the queue and mark them visited.
5. When the queue is empty, the traversal is complete. (For a disconnected graph, repeat from

every unvisited vertex.)

Vertex removed (visited)	Neighbours added	Queue after the step	Visited order so far
A	B, C	B, C	A
B	D	C, D	A B
C	E	D, E	A B C
D	F	E, F	A B C D
E	none	F	A B C D E
F	none	empty	A B C D E F

BFS order: A, B, C, D, E, F

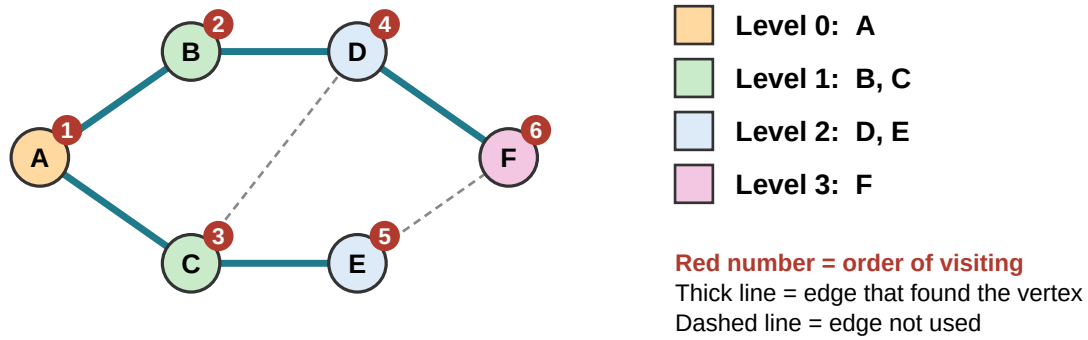


Figure 12: BFS from A: vertices are visited level by level

- **Time complexity:** O(V + E) with adjacency lists (O(V^2) with an adjacency matrix); **space:** O(V).
- BFS finds the **shortest path** (fewest edges) from the start vertex in an unweighted graph.
- **Applications:** shortest path in unweighted graphs, web crawling, social-network 'friends within k links', finding connected components.

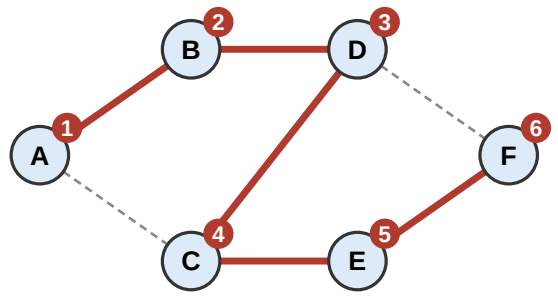
6.2 Depth First Search (DFS)

Definition: DFS starts at a given vertex, visits it, and goes as **deep** as possible along one path by moving to an unvisited neighbour. When no unvisited neighbour is left, it **backtracks** to the previous vertex. It uses a **stack** (or recursion).

Algorithm: DFS
1. Visit the start vertex, mark it visited and push it on the stack.
2. Repeat steps 3 and 4 while the stack is not empty.
3. Look at the vertex on top of the stack. If it has an **unvisited neighbour**, visit that neighbour, mark it visited and push it.
4. If it has **no unvisited neighbour**, pop it (backtrack).
5. When the stack is empty, the traversal is complete. (For a disconnected graph, repeat from every unvisited vertex.)

Action	Stack (bottom to top)	Visited order so far
Visit A	A	A
Visit B	A, B	A B
Visit D	A, B, D	A B D
Visit C	A, B, D, C	A B D C
Visit E	A, B, D, C, E	A B D C E
Visit F	A, B, D, C, E, F	A B D C E F
Backtrack from F	A, B, D, C, E	A B D C E F
Backtrack from E	A, B, D, C	A B D C E F
Backtrack from C	A, B, D	A B D C E F
Backtrack from D	A, B	A B D C E F
Backtrack from B	A	A B D C E F
Backtrack from A	empty	A B D C E F

DFS order: A, B, D, C, E, F



DFS order: A, B, D, C, E, F

Goes deep along one path:
A > B > D > C > E > F

then backtracks: F, E, C, D, B, A

Red number = order of visiting
Thick red line = edge used to go deeper
Dashed line = edge to a visited vertex

Figure 13: DFS from A: goes deep first, then backtracks

- **Time complexity:** O(V + E) with adjacency lists (O(V^2) with an adjacency matrix); **space:** O(V).
- DFS does not guarantee the shortest path.
- **Applications:** cycle detection, topological sorting, finding connected components, solving mazes and puzzles.

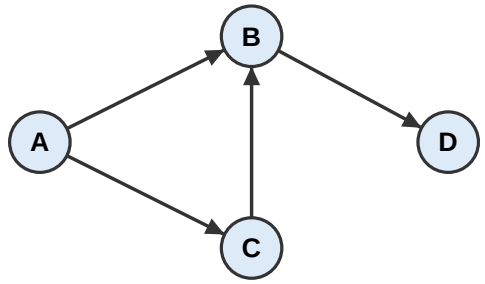
6.3 BFS vs DFS

Point	BFS	DFS
Data structure	Queue (FIFO)	Stack (LIFO) or recursion
Strategy	All neighbours first, level by level	Goes deep, then backtracks
Order for the example	A B C D E F	A B D C E F
Shortest path (unweighted)	Yes	No
Memory	Stores a whole level (more)	Stores one path (less)
Time	O(V + E)	O(V + E)
Uses	Shortest path, level order	Cycle detection, topological sort, mazes

7. Graph Representation

Answers Q6 (5 marks)

A graph is stored in two main ways: the **adjacency matrix** and the **adjacency list**. Both are shown for the undirected graph of Figure 9 and the directed graph of Figure 14.



Directed graph
Edges: A>B, A>C, C>B, B>D

Figure 14: A directed graph

7.1 Adjacency matrix

Definition: An **adjacency matrix** is a V x V two-dimensional array in which the entry in row i and column j is **1** if there is an edge from vertex i to vertex j, and **0** otherwise.

Undirected graph (Figure 9):

	A	B	C	D
A	0	1	1	0
B	1	0	1	0
C	1	1	0	1
D	0	0	1	0

Directed graph (Figure 14): row = from, column = to.

	A	B	C	D
A	0	1	1	0
B	0	0	0	1
C	0	1	0	0
D	0	0	0	0

- For an **undirected** graph the matrix is **symmetric**; row sum = degree of the vertex.
- For a **directed** graph it is not necessarily symmetric; row sum = out-degree, column sum = in-degree.
- For a **weighted** graph the weight is stored instead of 1 (and 0 or infinity where there is no edge).

Advantages	Disadvantages
Simple and easy to understand	Needs V x V memory even when there are few edges
Checking whether an edge exists takes O(1) time	Finding all neighbours of a vertex takes O(V)
Good for dense graphs (many edges)	Adding a vertex is costly; traversal takes $O(V^2)$

7.2 Adjacency list

Definition: An **adjacency list** stores, for every vertex, a **list of the vertices adjacent to it**. The graph is an array of V lists.

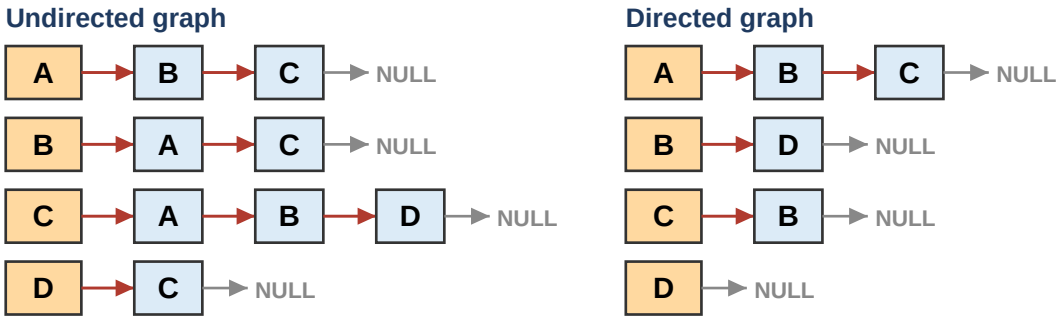


Figure 15: Adjacency lists of the undirected graph and the directed graph

- In an **undirected** graph each edge appears twice (once in the list of each end vertex).
- In a **directed** graph a vertex's list holds only the vertices it points **to**.
- For a **weighted** graph the weight is stored with each neighbour.

Advantages	Disadvantages
Needs only O(V + E) memory	Checking whether an edge exists takes O(degree) time
Finding the neighbours of a vertex is quick	Slightly harder to implement than a matrix
Good for sparse graphs (few edges); traversal takes O(V + E)	Not convenient for very dense graphs

7.3 Matrix vs list

Point	Adjacency matrix	Adjacency list
Structure	V x V table	Array of lists
Memory	$O(V^2)$	O(V + E)
Check an edge	O(1)	O(degree)
Find neighbours	O(V)	O(degree)
Best for	Dense graphs	Sparse graphs

8. How to Write Each Answer

Question	Write in this order
Q1 BST insertion and deletion (10)	Definition and property of a BST; insertion algorithm with the example and Figure 4; deletion algorithm with the three cases and Figures 5 to 7; complexity
Q2 Traversals (10)	Definition; the three orders (N L R, L N R, L R N); the three algorithms; the example tree with the results; uses; O(n)
Q3 Heap and heap sort (5)	Definition of heap, max-heap and min-heap with a diagram; heap sort algorithm; short example; O(n log n)
Q4 Terminology (5)	Draw the tree of Figure 1 and explain root, parent, child, sibling, leaf, internal node, degree, level, height
Q5 Graphs, BFS, DFS (10)	Definition; types with small diagrams; BFS and DFS definitions, algorithms, example with the order, complexity
Q6 Representation (5)	Adjacency matrix with example; adjacency list with example; advantages and disadvantages or a comparison
Q7 Height and degree (3)	Two definitions with one example from a diagram
Q8 Root node (2)	Topmost node, no parent, one per tree, starting point
Q9 Construct a BST (10)	Table of insertions with comparisons; final diagram; preorder, inorder and postorder results